淡江大學電機工程學系

專題實驗報告

SwinIR 超解析硬體加速器

SwinIR Superresolution Hardware Accelerator

指導教授：江正雄 博士

專題生：賴敬予

中華民國 115 年 9 月

目錄

第一章　緒論 **1**

1.1 摘要 1

1.2 研究背景與動機 1

1.3 實驗設計 2

1.4 論文架構 2

第二章　超解析硬體加速器簡介 **3**

2.1 傳統演算法 3

2.2 超解析技術 3

2.3 SwinIR 4

2.4 WMSA、SWMSA 4

2.5 超解析硬體加速器 5

2.6 評估指標介紹 5

2.6.1 超解析影像評估指標 5

I. 峰值訊噪比 (Peak signal-to-noise ratio, PSNR) 5

II. 結構相似性 (Structural Similarity, SSIM) 6

III. Difference Map 6

IV. Top-1 / Top-5 Accuracy 6

V. Precision / Recall / F1 6

2.6.2 硬體加速器評估指標 7

I. 吞吐量 (Throughput) 7

II. 時脈速率 (Clock Speed) 7

III. 功率 (Power) 7

IV. 延遲 (Latency) 7

V. 精準度(Accurancy) 8

VI. FPS (Frames Per Second) 8

VII. 硬體使用率(Utilization) 8

VIII. 推論時間(Inference Time) 8

2.7 系統搭載平台 8

2.7.1 軟體 8

I. Vivado 8

II. Jupyter Notebook 8

III. Vscode 8
IV. Petalinux 9
V. Vitis 9
2.7.2 硬體 9
I. ZCU106-Zynq UltraScale+ MPSoC 9
II. CPU、GPU 9
第三章 專題電路簡介 10
3.1 專題架構說明 10
3.1.1 專題神經網路架構 10
3.1.2 共享參數(Share Parameter) 10
3.2 專題電路架構說明 11
I. 整體電路簡介 11
II. CONV Unit 12
III. BatchNorm 14
IV. FC Unit 15
V. Residual 16
VI. SA Unit 16
第四章 專題結果展示 17
4.1 本專題模型訓練結果 17
4.2 硬體推論時間 18
4.3 RTL 電路模擬結果 18
I. CONV Unit 19
II. BatchNorm 20
III. FC Unit 21
IV. Residual 23
V. SA Unit 24
VI. Control Unit 25
4.3 HLS 電路模擬結果 26
I. CONV Unit (Row Stationary) 26
II. CONV Unit (Stream-based Linebuffer) 27
III. BatchNorm 28
IV. FC Unit 29
V. Residual 30
VI. SA Unit 31

VII. Control Unit 32
第五章　結論與未來展望 **34**
參考資料 **34**

圖目錄

圖一、超解析影像與插值法影像比較圖
圖二、SwinIR 架構圖
圖三、本專題神經網路架構
圖四、SwinIR 超解析硬體加速器 Block Diagram
圖五、Stream-based Linebuffer Convolution 架構圖
圖六、Linebuffer 填充(C code)
圖七、Linebuffer 填充(C code)
圖八、LeakyReLU 函數($a = 0.01$)
圖九、牛頓法迭代除法(C code)
圖十、牛頓法迭代除法結果(C code)
圖十一、牛頓法迭代根號(C code)
圖十二、牛頓法迭代根號結果(C code)
圖十三、本次專題的 GELU LUT 函數($x \leq 0$部分)
圖十四、SA Unit 架構圖
圖十五、專題模型推論結果與 Swin IR 模型推論結果比較
圖十六、專題模型推論結果
圖十七、Conv Unit 模擬結果(W=32, H=32, K=3, S=1)
圖十八、Conv Unit Utilization
圖十九、BatchNorm 與 Golden Pattern 比較結果
圖二十、BatchNorm 模擬結果(B=2, H×W=1024, C=96)
圖二十一、BatchNorm Utilization
圖二十二、FC Unit 模擬結果(W=1024, H=96, F=1024, E=192)
圖二十三、FC Unit Utilization
圖二十四、Residual 模擬結果(W=32, H=32)
圖二十五、Residual Utilization
圖二十六、SA Unit 模擬結果(W=8, H=8, R=2)
圖二十七、SA Unit Utilization
圖二十八、Control Unit Utilization
圖二十九、Conv Unit (Row Stationary) Block Design
圖三十、CONV Unit (Row Stationary) Utilization
圖三十一、CONV Unit (Row Stationary) Power Analysis
圖三十二、Conv Unit (Stream-Based Linebuffer) Block Design
圖三十三、CONV Unit (Stream-Based Linebuffer) Utilization
圖三十四、CONV Unit (Stream-Based Linebuffer) Power Analysis
圖三十五、BatchNorm Block Design
圖三十六、BatchNorm Utilization

圖三十七、BatchNorm Power Analysis
圖三十八、FC Unit Block Design
圖三十九、FC Unit Utilization
圖四十、FC Unit Power Analysis
圖四十一、Residual Block Design
圖四十二、Residual Utilization
圖四十三、Residual Power Analysis
圖四十四、SA Unit Block Design
圖四十五、SA Unit Utilization
圖四十六、SA Unit Power Analysis
圖四十七、Control Unit Block Design
圖四十八、Control Unit Utilization
圖四十九、Control Unit Power Analysis

表目錄

表一、SwinIR 硬體硬體速器共享參數(Share Parameter)
表二、SwinIR 硬體硬體速器各層資訊
表三、專題模型訓練結果與 Swin IR 模型訓練結果比較
表四、YOLO v5 在不同放大方法下的推論結果比較(X2 放大倍率)
表五、GPU 與 CPU 在 SwinIR 模型的推論速度
表六、Conv Unit 模擬結果(1 個C_{conv})
表七、Conv Unit 量測結果(1 個C_{conv})
表八、Conv Unit + LeakyReLU Fused Layer 模擬結果
表九、BatchNorm 模擬結果
表十、BatchNorm 量測結果
表十一、FC Unit 模擬結果
表十二、FC Unit 量測結果
表十三、FC Unit +GeLU Fused Layer 模擬結果
表十四、Residual 模擬結果
表十五、Residual 量測結果
表十六、SA Unit 模擬結果
表十七、SA Unit 量測結果
表十八、Control Unit 量測結果
表十九、Conv Unit (Row Stationary) HLS 量測結果
表二十、Convolution 比較結果
表二十一、Conv Unit Stream-Based Linebuffer) HLS 量測結果
表二十二、BatchNorm HLS 量測結果
表二十三、FC Unit HLS 量測結果
表二十四、Residual HLS 量測結果
表二十五、SA Unit HLS 量測結果
表二十六、Control Unit HLS 量測結果
表二十七、定點數最佳化 SwinIR 超解析硬體加速器與 QFSRCNN 結果比較表

第一章 緒論

1.1 摘要

影像超解析（Super-Resolution, SR）技術在電腦視覺應用中扮演關鍵角色，對於提升影像品質及增進後續高層次任務（如 YOLO 影像辨識、語意分割等）之準確性具有顯著助益。近年來，基於 Transformer 架構的 SR 模型顯示出卓越的重建性能，其中 SwinIR（Swin Transformer for Image Restoration）模型因具備層次化架構與移動式視窗自注意力（Shifted Window MSA）機制，於多項影像復原任務中展現領先成效。然而，SwinIR 模型在運算與記憶體資源方面需求較高，對即時處理與嵌入式系統應用造成實質限制。

本研究目的在針對 SwinIR 模型設計一套具可擴展性與高效率的硬體加速架構，實現在 FPGA 平台上之即時 SR 推論系統。設計上，採模組化思維進行硬體架構重構，並提供多項可調參數，包括特徵維度、視窗多頭自注意力（W-MSA 與 SW-MSA）之 head 數量、網路深度並行度與 pipeline 階段等，以因應不同應用場景中對運算效能、推論延遲及資源使用的需求。此外，本加速器導入資料流優化技術，特別設計行緩衝器（Line Buffer），有效降低外部記憶體存取頻率，進而提升資料處理效率並減少延遲。

為驗證所提架構之效能與可行性，本研究將系統實作於 Xilinx ZCU106（Zynq UltraScale+ MPSoC）開發平台，進行實測驗證，評估其在影像重建品質（以 PSNR 為指標）。實驗結果顯示，本加速器不僅可實現即時之 SR 推論，亦具備高度適應性與移植性，適合應用於資源受限之邊緣裝置與智慧視覺系統。

1.2 研究背景與動機

隨著人工智慧與深度學習的快速發展，電腦視覺應用已廣泛應用於監控系統、自駕車、智慧醫療、無人機導航等領域。然而，在實際場景中，受限於攝影設備、傳輸頻寬或儲存容量，取得的影像常為低解析度，進而導致辨識錯誤、分類困難、語意分割不盡理想等問題，降低整體系統效能。SR 的出現正是為了回應這項需求，也有多項研究已證明，在物件檢測等任務中加入 SR 作為前處理，可有效提升準確率，對衛星影像中車輛等小物件的偵測提升幅度可達 13~36%[2]，此外，對分類任務的實驗亦顯示，使用 VDSR 等超解析前處理可恢復 80 %的精度[3]。由此可見，SR 技術在提升低解析度影像品質並加強下游任務效能方面具有顯著價值。

過去已有多種針對 SR 的硬體加速器設計展現良好性能，例如:SESR[4]得到了 26.65 的 PSNR，透過改良的 DCNNs[5]更是達到了 36.40 的 PSNR 的好結果。然而，這些設計多數未涵蓋完整的標準驗證資料集，仍有提升空間。因此，本次的設計將以 SwinIR 為核心在 Set5,Set14,BSD100,Urban100,Manga109 等標準影像驗證集進行完整測試與優化。將參考改良 DCNNs[5]的 PSNR 評估標準選擇以浮點數作為本次的主要計算單位，並嘗試挑戰目前 SR 硬體加速器與 CPU、GPU 推論所能達成的最高 PSNR，力求在精度、效能與功耗上取得突破性成果。

1.3 實驗設計

本次的專題架構會分為兩種設計方法，分別為使用傳統的 RTL 進行浮點數的可擴增(Scalable)以及使用 HLS 轉為 RTL 進行定點數計算，整合成兩種設計電路分別為浮點數可擴增 SwinIR 超解析硬體加速器，定點數最佳化 SwinIR 超解析硬體加速器，在神經網路、硬體架構相同，兩者採用的測試方式不同。

RTL 設計採用 RTL Testbench 進行測試，使用 Python 程式碼製作輸入資料、Golden Test Pattern，並使用 Testbench 輸入資料到 RTL 硬體加速器，輸出資料文件，並讓 Python 進行準確度驗證，再將電路進行約束(Constraint)並進行 Synthesis 與 Implementation 電路效能驗證。

HLS 設計採用 C Testbench 進行測試，使用 C 程式碼製作輸入資料、Golden Test Pattern，並使用 Testbench 輸入資料到 C code 硬體加速器，輸出在 C Testbench 內進行準確度驗證，最後在將 RTL/C Cosimulation 結束後轉為 RTL 並進行 Synthesis 與 Implementation 電路效能驗證，並利用進行 FPGA 進行驗證。

最後將設計進行 FPGA 的驗證，並把硬體加速器的推論結果與現有的超解析硬體加速器研究、GPU、CPU 進行效能包刮 Utilization、Power、Clock Speed、FPS 的比對。

1.4 論文架構

本專題論文分為五個章節，第一章為緒論，第二章為超解析硬體加速器簡介，第三章為專題電路簡介，介紹本專題 SwinIR 運作原理與本專題使用之電路，第四章專題結果展示針對本專題電路進行 RTL 子電路以及 HLS 電路的結果展示，第五章為結論與未來展望。

第二章 超解析硬體加速器簡介

2.1 傳統演算法

在影像處理領域中時常會在放大影像時出現馬賽克效應嚴重的情況，而為了解決這樣的問題影像解析度不符合預期的情況。為了解決這樣的問題，出現了許多影像提升的演算法，例如:最近鄰插值法(Nearest neighbor interpolation)、雙線性插值法(Bilinear interpolation)、雙三次插值法(Bicubic interpolation)等。這些傳統的插值算法通過周圍像素的值來猜測並填充放大後產生的新畫素位置。最近鄰插值法直接複製最相近畫素的值，導致放大後產生明顯的鋸齒狀邊緣，雙線性插值和雙三次插值則通過計算相鄰畫素的加權平均來使過渡更平滑，雖然有效地減少了鋸齒，但結果往往是一張更模糊的影像。這是因為算法並不知道這些畫素原本應該是什麼，它只能根據現有的有限資訊進行平滑的推測，也無法創造影像中原本就不存在的資訊，只能以犧牲銳利度為代價來換取更大的影像尺寸。

2.2 超解析技術

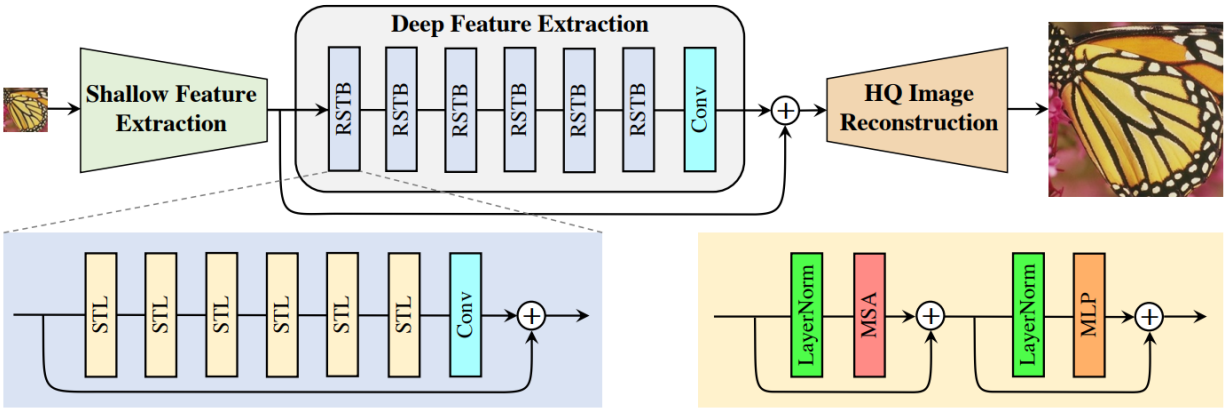
為從根本上突破傳統演算法的限制，超解析度成像技術應運而生。其核心思想是從一組或多張低解析度影像中，重建出高解析度影像。隨著深度學習的成長，出現了 SRCNN [10] (Super-Resolution Convolutional Neural Network) 證明了僅有三層的簡單 CNN 其效果也能大幅超越傳統的插值方法，SRGAN (Super-Resolution Generative Adversarial Network) [11] 加入了對抗訓練(Adversarial Training)，生成視覺上更自然、紋理更清晰的影像，SwinIR [1]的 Transformer-based 神經網路架構更是將影像的品質帶到了更高的階級。未來面臨影像的低解析度問題，SR 即是問題的解法首選，SR 具備能夠重建出超越感光元件與光學系統物理極限，甚至超越原圖的高解析圖，能夠真正輔助並提升各個影像處理任務表現的重要技術。



圖一、超解析影像與插值法影像比較圖

2.3 SwinIR

SwinIR 作為 Swin Transformer[6]的改良，針對了 SR 的任務進行了調整。如圖一所示，SwinIR 整體架構可分為三個主要區塊:淺層特徵提取(Shallow Feature Extraction)、深層特徵提取(Deep Feature Extraction)和高品質影像重建(High Quality Image Reconstruction)。這三個區塊的神經網路架構處理使 SwinIR 的超解析在現今超解析影像的架構中仍然佔有相當高的地位，PSNR 最高可以抵達 38.46，使 SwinIR 成為了重要的 Transformer-Based 超解析架構。



(a) Residual Swin Transformer Block (RSTB)　　(b) Swin Transformer Layer (STL)

圖二、SwinIR 架構圖［1］

2.4 WMSA、SWMSA

SwinIR 中深層處理使用了 Window-based Multi-head Self-Attention(WMSA)、Shifted Window-based Multi-head Self-Attention(SWMSA)兩個架構交換使用處理進階的特徵提取，其概念利用了 Self-Attention。Self-Attention [7]的原理是將輸入特徵 X 進行線性投影產生 Query（Q）、Key（K）、Value（V），利用 Q 與 K 的點積除以 Q、K 的維度 d，再經過 Softmax 計算 Attention Score（A'），最後透過A'與 V 的矩陣相乘得到輸出特徵。MSA 的原理則是透過將輸入特徵切為多個 head，並讓每個 head 都分別進行 Self-Attention，再將結果串接起來作為輸出。WSMA、SWMSA 則引入了視窗的概念，單獨對各個視窗進行 MSA，這樣的方式不只可以降低計算的複雜度，同時增加了局部特徵的建模能力。然而，WMSA 僅限於視窗內部運算，無法捕捉視窗之間的跨區域特徵。為解決此問題，SWMSA 採用了視窗位移策略，將視窗偏移一半長寬，讓特徵區塊跨越原本的視窗邊界，並透過遮罩機制修正因位移造成的 Query 與 Key 配對錯誤，進一步提升全域建模能力。此外，為了更準確地捕捉區域內相對位置資訊，WMSA、SWMSA 在 Self-Attention 中加入了位置相關偏差(Relative Position Bias)使A'可以包含視窗內位置的資訊，讓模型能夠辨識同一視窗中不同位置的相對關係。

WMSA、SWMSA 內 Self-Attention 的計算方式:

$$Q = W^q X, K = W^k X, V = W^v X \ ,(1)$$

$$A' = Softmax\left(\frac{QK^T}{\sqrt{d}} + Relative\ Position\ Bias\right) \ ,(2)$$

$$O = VA' \ (3)$$

2.5 超解析硬體加速器

超解析的模型在訓練階段大量依靠 GPU 的併行運算處理大量的影像資料，在訓練階段 GPU 的能力固然強大，但是在推論階段，影像資料少的情況下，使用 GPU 反而會使推論速度降低，這是因為 GPU 進行推論會因為需要進行設備與主機間內存映射的協調，這樣的初始化的時間使 GPU 進行推論時執行速度大幅降低。因此進行超解析的推論需要依靠 FPGA 的加速器進行硬體的加速，透過 FPGA 低延遲與低功率的特性，可以使 FPGA 的能效比(TOPS/W)高出 GPU 幾個數量級，也因為 FPGA 的確定性運算，解決了通用處理器的執行時間容易受到後台任務、作業系統調度、記憶體存取波動等因素影響，導致延遲不穩定（Jitter）的影響，這使得在邊緣設備（如手機、無人機、攝影機等等）上實時運行高品質超解析度成為可能。

2.6 評估指標介紹

2.6.1 超解析影像評估指標

在影像處理領域中，如何客觀地評估處理後的影像與原始影像之間的差異至關重要。其中，峰值訊噪比 (Peak Signal-to-Noise Ratio, PSNR) 和結構相似性 (Structural Similarity, SSIM) 是兩種最常用且重要的全參考式 (Full-Reference) 影像品質評估指標，Difference Map 則是能可視化顯示圖像間的差異。

I. 峰值訊噪比 (Peak signal-to-noise ratio, PSNR)

峰值訊噪比透過計算壓縮或處理後的影像與原始影像之間的平均平方誤差 (Mean Squared Error, MSE) 來量化兩張影像的像素級差異。PSNR 的數值以分貝 (dB) 為單位，範圍再 0~50 之間，數值越高，代表兩張影像的差異越小，即還原的影像品質越接近原始影像，反之，數值越低則表示失真越嚴重。PSNR 的計算簡單且物理意義明確，但其主要缺點在於它僅基於像素值的誤差進行計算，與人類視覺系統 (Human Visual System, HVS) 對影像品質的感知並非完全一致。

PSNR 計算公式如下：

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, (1)$$

$$\text{PSNR} = 10log_{10}\left(\frac{MAX^2}{MSE}\right) \quad (2)$$

II. 結構相似性 (Structural Similarity, SSIM)

為了解決 PSNR 與人眼主觀感受不符的問題，於是 SSIM 被提出。SSIM 的核心思想在於人眼視覺對影像的結構資訊非常敏感而非辨識出獨立的像素誤差。SSIM 從亮度 (Luminance)、對比度 (Contrast) 和結構 (Structure) 三個面向來比較兩張影像的相似性。SSIM 的範圍再 0~1 之間，SSIM 越接近 1 越好，反之，數值越低則表示越不像原影像。

SSIM 計算公式如下:

$$l(x,y) = \frac{2\mu_x\mu_y + C_1}{\mu_x^2 + \mu_y^2 + C_1}, (1)$$

$$\mathrm{c(x,y)} = \frac{2\sigma_x\sigma_y + C_2}{\sigma_x^2 + \sigma_y^2 + C_2}, (2)$$

$$s(x,y) = \frac{\sigma_{xy} + 0.5C_2}{\sigma_x\sigma_y + 0.5C_2}, (3)$$

$$\text{SSIM} = l(x,y) \times c(x,y) \times s(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} (4)$$

III. Difference Map

Differnece Map 的評估方式是將兩張對齊的圖像 A 和圖 B 逐像素比對，計算出像素的差異 D，然後用顏色映射可視化出來，本次專題使用絕對誤差進行計算。

Difference Map 計算公式如下:

$$\mathrm{D}(x,y) = |\mathrm{A}(x,y) - \mathrm{B}(x,y)|$$

IV. Top-1 / Top-5 Accuracy

Top-1 Accuracy 為評估 yolo 分類模型中機率最高的類別是否正確，Top-5 Accuracy 則為真實標籤是否出現在模型預測的前 5 名中。

V. Precision / Recall / F1

Precision 為預測為某類時，預測中有多少為真。Recall 為真正屬於某類的樣本中，有多少被預測出來。F1-score 則是 Precision 和 Recall 的調和平均。

Precision / Recall / F1 計算公式如下:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

2.6.2 硬體加速器評估指標

硬體加速器的評估標準包含了吞吐量(Throughput)、時脈速率(Clock Speed)、功率(Power)、延遲(Latency)、精準度(Accurancy)、FPS(Frame Per Second)

I. 吞吐量 (Throughput)

用於衡量硬體加速器的乘加運算(Multiply–accumulate, MAC)能力，代表了加速器的整體處理能力。高吞吐量意味著加速器能在一定時間內處理更多幀（Frame）的影像完成更多計算任務，常用 TOPS (Tera-Operations Per Second, 每秒萬億次操作)、GOPS (Giga-Operations Per Second, 每秒十億次操作)進行表達。

II. 時脈速率 (Clock Speed)

處理器內部同步電路的工作頻率，即每秒鐘有多少個時脈週期，時脈速率決定了電路基本單元的執行速度。在架構相似的情況下，更高的時脈通常意味著更快的計算速度。常用單位 MHz 或 GHz 表達。
Clock Speed 計算公式如下:

$$Clock\ Speed = 1/\big(Clock\ Period - WNS(Worst\ Negative\ Slack)\big)$$

III. 功率 (Power)

整體硬體加速器在推論進行時所消耗的總能量速率。常用單位瓦特(Watt, W)表達，是採用 FPGA 的關鍵要點，因為在功耗上 FPGA 相較於 GPU 可以達成較少的功耗的特性，使 FPGA 在應用上較 GPU 可以更加不受限制。

IV. 延遲 (Latency)

延遲代表著從輸入數據進入加速器到處理後的結果輸出所經歷的總時間也即系統的反應速度，對於即時交互的應用至關重要。常用單位毫秒(ms)、微秒(μs)、奈秒(ns)表達。

V. 精準度 (Accurancy)

硬體加速器中輸出的數值與預期數值的差距，決定了加速器輸出結果的質量。如果一個加速器速度極快、功耗極低，但輸出的影像滿是噪點和失真，那麼它也毫無用處，本次的硬體加速器將採用浮點數 FP32 的標準進行設計，在提高 PSNR、SSIM 的品質。

VI. FPS (Frame Per Second)

FPS 代表每秒幀數，直接決定了硬體加速器產生圖片的速度，是衡量產生圖片速度的重要指標。

VII. 硬體使用率(Utilization)

硬體實現時使用到的元件與數量，包含用來實現組合邏輯與部分暫存器的 LUT 元件、用於乘法器、乘加結構等數學運算的 DSP、IO 與外部介面的接腳、BUFG 全域時鐘網路驅動器、用於資料緩衝、權重儲存與大型記憶體 BRAM 等。

VIII. 推論時間(Inference Time)

表達不同硬體與模型進行推論時，所需要消耗的時間，分成 SwinIR 原模型的 GPU、CPU 表現以及 SwinIR(This Work)表現三個比較。

2.7 系統搭載平台

2.7.1 軟體

I. Vivado:

進行 RTL code 編譯、Simulation、Synthesis、Implementation、Bitstream、Block Diagram 的產生與最後硬體設計的匯出。

II. Jupyter Notebook:

進行 HLS 的編譯，以操作 FPGA 的硬體，透過 AXI Bus 傳輸資料控制資料流，並利用 DMA 進行讀寫資料，測試硬體加速模組功能。CPU、GPU 的 SwinIR 推論也會在這裡進行。

III. Vscode:

撰寫 RTL code、各個模組的 C code，SwinIR 的訓練也在這裡進行。

IV. Petalinux:

嵌入式 Linux 開發工具，建立 Bootable Linux 映像，驅動開發與配置並整合硬體 IP 與應用程式。

V. Vitis

在 HLS 階段將撰寫好的 C code 進行 C Simulation、Synthesis、C/RTL CoSimulation 最後轉為 RTL 並在 vivado 進行 block Design 的設計。

2.7.2 硬體

I. ZCU106-Zynq UltraScale+ MPSoC:

整合 PS 與 PL 端功能進行硬體加速器在 FPGA 上的驗證。

II. CPU、GPU:

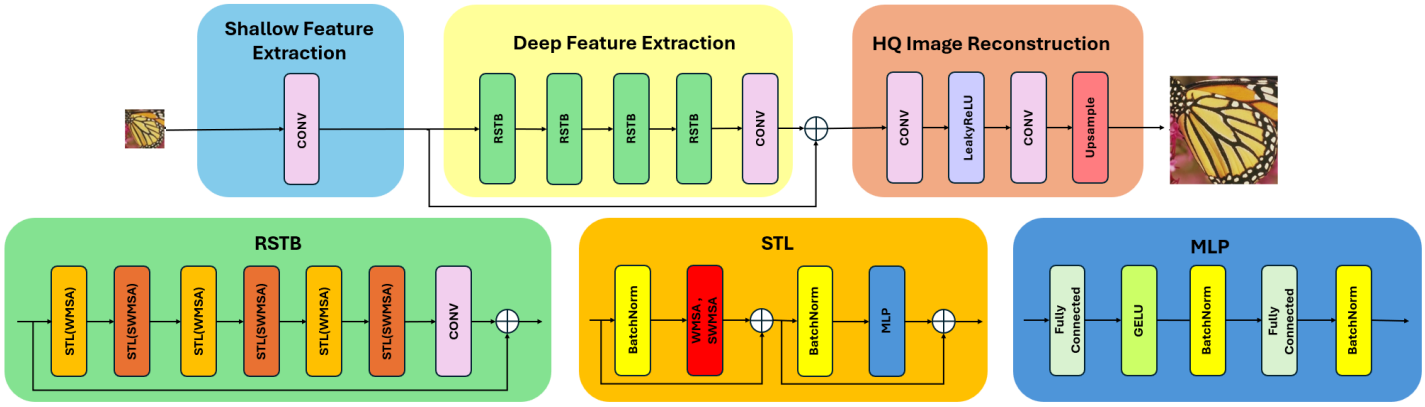
CPU 使用 Intel i7-11700，GPU 則是 NVIDIA Geforce RTX 3090

第三章 專題電路簡介

3.1 專題架構說明

3.1.1 專題神經網路架構

SwinIR 雖然以高解析度的影像還原而聞名，但是 SwinIR 推論使用 GPU 或 CPU 無法有效滿足實時處理需求，因此，本次的設計將其轉化為硬體架構以解決無法實時呈現 SR 結果的問題。SwinIR 超解析硬體加速器保留了淺層特徵提取的設計，在深層特徵提取的部分，將原有的六次 RSTB 剪枝(Pruning)為四次 RSTB 減少推論的時間。另外，參考文獻[9]將 STL 中 LayerNorm 的部分改為 BatchNorm，並在 MLP 中兩個全連接層的計算後分別加入 BatchNorm 使計算的同時能保留特徵計算的能力也能減少計算量。最後，高品質影像還原的部分，將 Upsmaple 的部分改由軟體執行以避免其不友善的硬體計算，本專題神經網路架構如圖三所示。



圖三、本專題神經網路架構

3.1.2 共享參數(Share Parameter)

本專題浮點數可擴增 SwinIR 超解析硬體加速器設有共享參數可以對所有模組進行神經網路架構的調整，由於模組皆具備可調整參數的設計，因此可以彈性設定特徵維度、卷積核、步距、MSA 的 head 數量、Parallel 數量、Pipeline 數量、以及 SR 放大倍率等，能根據不同應用場景將運算速度、精度或功耗等需求調整配置，提升系統的適應性與擴展性，實現即時的 SR 可改架構的最佳化推論系統。

3.2 專題電路架構說明

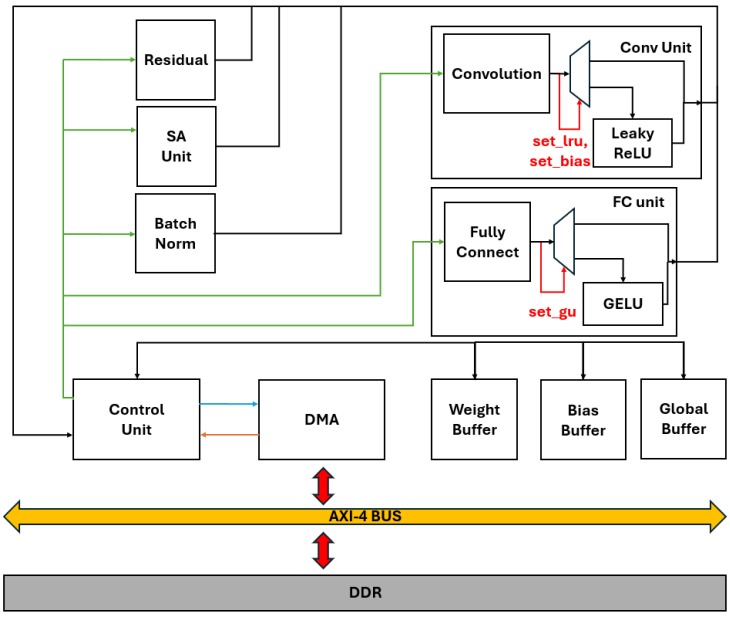
I. 整體電路簡介

本次設計將預特徵提取、主要特徵提取和高品質影像重建三個區塊由 Conv Unit、SA Unit、BatchNorm、FC Unit、Residual 五個模組組成，透過 Control Unit 對所有單元進行控制以及資料的輸入、輸出，DMA 則負責 DDR 內資料的讀取以及寫入。Weight Buffer 儲存了卷積核、全連接層、Self-Attention 線性投射Q、K、V的權重，Bias Buffer 則儲存了卷積的偏差(Bias)以及 WMSA/SWMSA 的 relative bias，Global Buffer 則用於 BatchNorm 的計算以及記憶用於 Residual 的矩陣。另外，在浮點數可擴增設計下全部的硬體模組都可以透過改變表一中的共享參數自由切換網路內部架構，而表二中的各層輸入、輸出也會隨之變動，因此，表二紀錄目前記錄的是輸入維度B、C、H、W為 1、96、32、32 的情況。本次設計的硬體加速器架構如圖四所示。

Share Parameter	Description
D	data width
H/W	ifmap height/width
E/F	ofmap height/width
K	filter height & width
S	filter stride
M	WMSA、SWMSA Window height & width
h	# of WMSA's and SWMSA's head
C	# of Ifmap & filter channel
P	# of calculate parallel
p	# of calculate pipeline
B	# of Batch
R	output resolution
T	# of Taylor Series

Layer		Input (C H W)	Weight	Output (C H W)
預特徵提取	Conv	3, 32, 32	3,1,3 (K,S,C) x96	96, 32, 32
主要特徵提取	Conv	96, 32, 32	3,1,96 (K,S,C) x96	96, 32 32
	BatchNorm	96, 32, 32	-	96, 32, 32
	SA Unit	96, 1024 (C HxW)	96,288 (H,W)	96, 1024
	FC Unit	96, 1024	First 96,192 (H,W) Second 192, 96	First 192,1024 (H,W) Second 96, 1024
	Residual	96, 32, 32	-	96, 32, 32
高品質影像重建	Conv1	96, 32, 32	3,1,96 (K,S,C) x64	64, 32 32
	Upsample	64, 32, 32	-	64, 64, 64
	Conv2	64, 64, 64	3,1,64 (K,S,C) x3	3, 64, 64

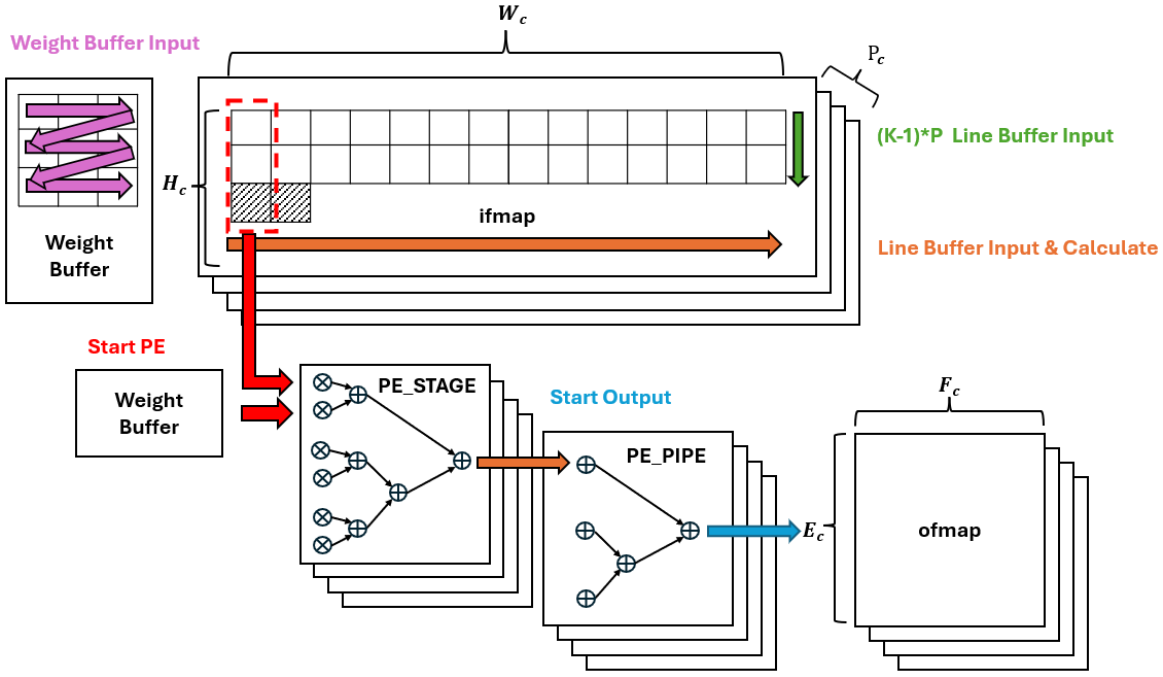
表一、二 SwinIR 硬體硬體速器共享參數(Share Parameter)、各層資訊



圖四、SwinIR 超解析硬體加速器 Block Diagram

II. CONV Unit

本次的專題架構內包含八層的卷積運算與多層的 WMSA、SWMSA 計算，考慮到如此龐大的計算量，如果採用 Eyeriss[12]的 Row Stationary 計算進行卷積計算，在卷積核的移動計算累積 Pre Sum 會造成大量的功耗，於是為了避免這樣的情況，參考文獻[8]在卷積運算中採用了 Stream-based Linebuffer 的設計，這樣的設計可以使卷積計算時在一個C_{conv}的情況下透過$K-1$個 Line Buffer 進行 ifmap 資訊的儲存的同時進行卷積的計算，讓模組可以使用最少的功耗並維持計算與輸入的效率。卷積運作如圖三所示，首先是卷積計算的部分，內部的模組 PE_STAGE 的功能會透過 Addder Tree 加總卷積核內K個 PE 乘積結果，PE_PIPE 則是選擇了在輸入特徵的S步後進行 PE_STAGE 的 Pipeline 並隨著開始的點前進K步計算卷積和，根據K決定p_{conv}的數量。透過 PE_STAGE、PE_PIPE 可以將卷積和正確的輸出並支援同時間多 Pipeline 設計，以及保持多 Parallel 的進行。另外，可以在各層的卷積運算彈性調整H_{conv}、W_{conv}、K、S的功能，可以根據需求彈性調整。另外，在最後高品質影像重建的部份，透過 Control Unit 輸出的 set_lru 訊號，可以選擇是否與 LeakyReLU 激勵函數做成 Fused Layer 這樣一來一旦卷積計算完成，就可以進行 LeakyReLU 的計算，並緊接著下一層卷積的開始，使計算的時間近一步的縮短也減少了 On-chip memory 的消耗。最後一點，由於 BatchNorm 的計算包含了平均數相減，所以卷積計算可以省略 bias 的計算，配合可以判斷 Conv Unit 是否處於 BatchNorm 後的 set_bias 訊號，可以決定是否加上 bias，以刪去不必要的計算使速度加快。

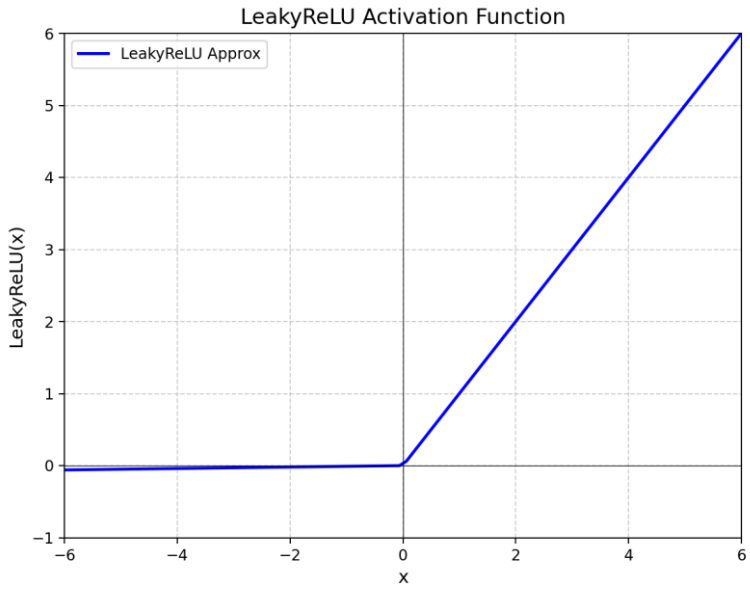


圖五、Stream-based Linebuffer Convolution 架構圖

```
// 初始化 Line Buffer
for (int i = 0; i < kernel_height; i++) {
    for (int j = 0; j < input_width; j++) {
        if (i < kernel_height - 1) {
            // 前 kernel_height-1 行初始化為 0
            line_buffer[i * input_width + j] = 0.0f;
        } else {
            // 最後一行將從輸入圖像填充
            line_buffer[i * input_width + j] = input[j];
        }
    }
}
```

```
// 將 Line Buffer 向上移動一行
for (int k = 0; k < kernel_height - 1; k++) {
    for (int j = 0; j < input_width; j++) {
        line_buffer[k * input_width + j] = line_buffer[(k + 1) * input_width + j];
    }
}
// 將新的一行數據加載到 Line Buffer 的最後一行
int input_row = i + kernel_height - 1;
if (input_row < input_height) {
    for (int j = 0; j < input_width; j++) {
        line_buffer[(kernel_height - 1) * input_width + j] = input[input_row * input_width + j];
    }
}
```

圖六、七、Linebuffer 填充(C code)



圖八、LeakyReLU 函數($a = 0.01$)

III. BatchNorm

BatchNorm 計算了所有B中同一C中的平均數、標準差的計算並將整個 Batch 進行標準化，LayerNorm 則會對一個B中所有C的部分進行平均數、標準差的計算與標準化，底下為單次 BatchNorm、LayerNorm 的計算複雜度。由於推論階段各個模組內C的數量都較B多，多個 Layer 的存在，加上需要計算平均值、標準差的同時也需要消耗更多的 Global Buffer 資源，使 LayerNorm 對硬體的負擔較大。於是本次設計採用了 BatchNorm 代替 LayerNorm 的設計。如圖四所示，將 STL 原始架構中的 LayerNorm 替換為 BatchNorm，並於 MLP 中的兩層全連接層之後分別加入 BatchNorm，不只可以減少 Global Buffer 的消耗，也能保持原有的模型表現。

BatchNorm 與 LayerNorm 複雜度比較如下:

$$O(BatchNorm) = H_{Batch} \times W_{Batch} \times B, (1)$$
$$O(LayerNorm) = H \times W \times C \quad (2)$$

BatchNorm 在計算上需要對所有數據進行標準化，因此需要使用到浮點數的除法與根號的設計，除法的部分使用牛頓法(Newton's method)，首先要計算除數 b 的倒數，將除數的指數部分設置為 126 得到X_0，乘上初始的猜測常數$\frac{32}{17}$，用$\frac{48}{17} - \frac{32}{17}X_0$得到下一次的結果，在經過運算後$X_n \times (2 - B \times X_n)$迭代三次完成倒數 b，最後將被除數 a 乘上倒數 b 即可得到結果O。

```c
// Newton-Raphson reciprocal approximation on mantissa only
float reciprocal_newton(float mantissa) {
    // 初始近似值: 48/17 - (32/17)*mantissa
    float x = 48.0/17.0 - (32.0/17.0) * mantissa;
    for (int i = 0; i < 3; i++) {
        x = x * (2.0 - mantissa * x);
    }
    return x;
}
```

```
A = 7.500000, B = 2.500000
A/B = 3.000000
```

圖九、十 牛頓法迭代除法(C code)、結果(C code)

牛頓法迭代除法計算如下:

$$X_1 = \frac{48}{17} - \frac{32}{17}X_0, (1)$$
$$X_{n+1} = X_n \times (2 - B \times X_n), 3 \geq n > 0, B = 2, (2)$$
$$O = a \times X_4 \quad (3)$$

根號的部分需要使用牛頓法進行開根號，因為牛頓法迭代每次迭代正確位數大約翻倍。計算的開始要將原指數右移 1 位（除以 2）並加上 63，調整 IEEE 754 的指數偏移量得到X_0，將需要開根號的輸入值a除去X_0，加上X_0，最後除去 2 得到輸出值，接下來的迭代則是將這次的X_n換成上次的輸出值X_{n+1}，迭代四次即可得當正確的根號。

```
float half = 0.5;

// 迭代 3 次
for (int i = 0; i < 3; i++) {
    DIV.f = A.f / X.f;              // a / x
    ADD.f = DIV.f + X.f;            // a/x + x
    X.f   = ADD.f * half;           // 0.5*(a/x + x)
}
```

```
sqrt(16.000000) = 4.000000
sqrt(2.000000) = 1.414214
sqrt(0.250000) = 0.500000
```

圖十一、十二 牛頓法迭代根號計算、結果(C code)

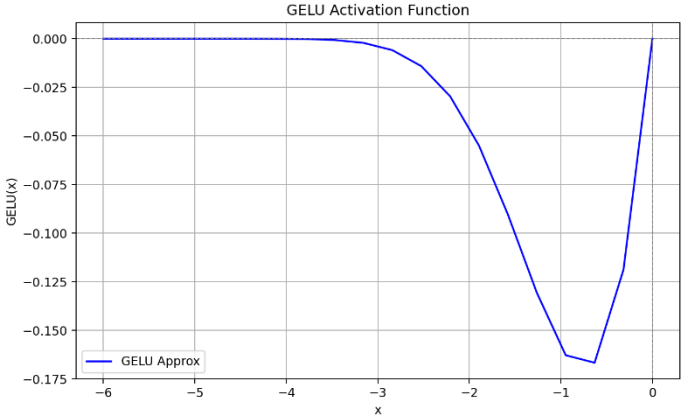
牛頓法迭代根號計算如下:

$$X_1 = \frac{(X_0 + a/X_0)}{2}, n = 0, (1)$$

$$X_{n+1} = \frac{(X_n + a/X_n)}{2}, 3 \geq n > 0$$

IV. FC Unit

同 Conv Unit 內配合激勵函數設計 Fused Layer 一樣，透過 Control Unit 控制 set_gu 判斷 MLP 中目前的全連接層位置，第一層，將全連接層與 LUT 製作成的 GELU 做成 Fused Layer，這樣一來全連接層一旦計算完成就可以進行 BatchNorm 的計算，第二層，則不使用 Fused Layer，透過 MLP 階層的判斷切換適合的全連接層設計，使計算的時間近一步的縮短也減少了 On-chip memory 的消耗。



圖十三、本次專題的 GELU LUT 函數($x \leq 0$部分)

V. Residual

Residual 透過 Global Buffer 儲存先前進行的操作得到的矩陣，在殘差連結的部分進行兩矩陣的相加，對 Global Buffer 的消耗較大，該模組設計用於與要進行殘差連結前的模組進行 Pipeline 計算，使計算速度可以進一步提高。

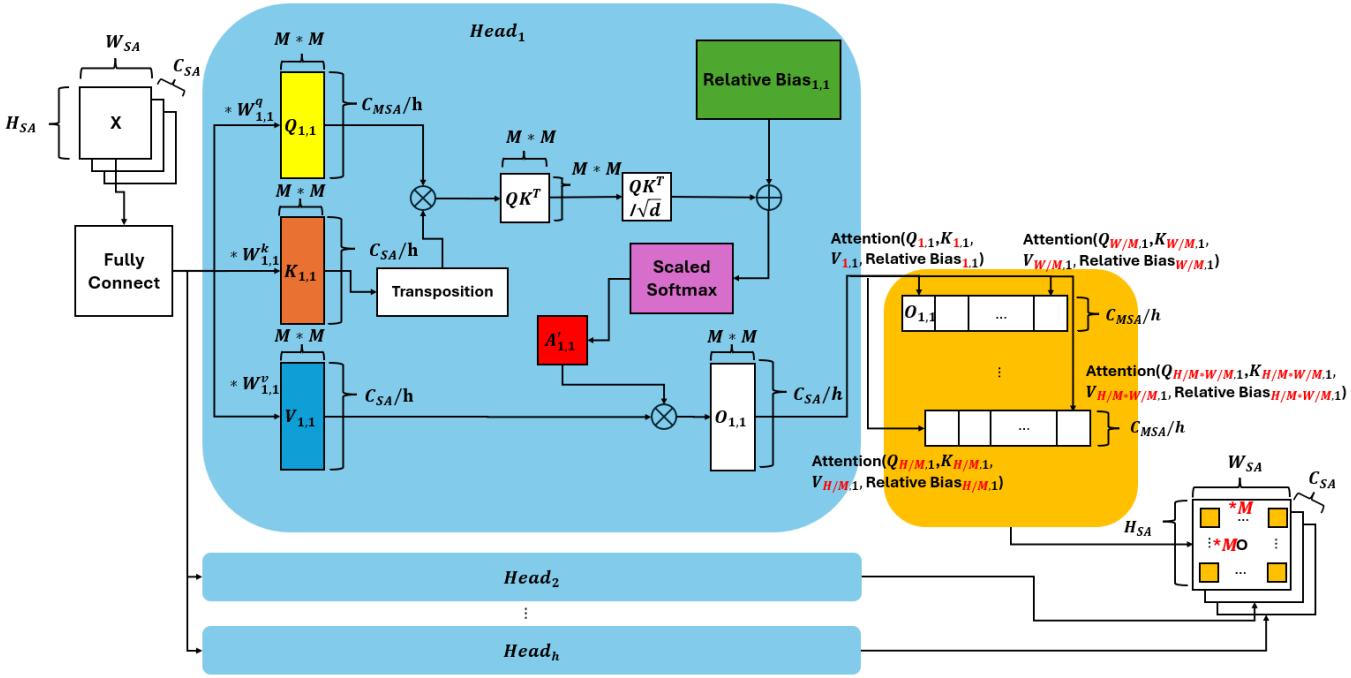
VI. SA Unit

本次設計了 SA Unit 作為可重複使用的 Self-Attention 運算單元，供所有 head 共用，依照 Self-Attention 計算方式，可以自由調整H_{WMSA}、W_{WMSA}、M、R。另外，在 SA Unit 中會使用到 Softmax 的計算方法，而 Softmax 進行計算會需要使用浮點數的指數運算，進行指數運算的方法使用泰勒展開進行設計，因此需要非常長的時間進行計算，所以本專題另外設計 Scaled Softmax 針對 SWMSA 中遮罩造成的 Softmax 值接近 0 的情況，設計中加入遇到特定的 ifmap 數值可直接將該位置的$A^{'}$數值設為 0，跳過 Softmax 計算，從而減少不必要的運算資源浪費。SA Unit 的架構如圖八所示。

Softmax 函數計算方法如下:

$$e^x = \sum_{j=0}^{n} \frac{x^j}{j!}, \quad (1)$$

$$Softmax(x_i) = \frac{e^{x_i}}{\sum_{j=0}^{n} e^{x_j}}$$



圖十四、SA Unit 架構圖

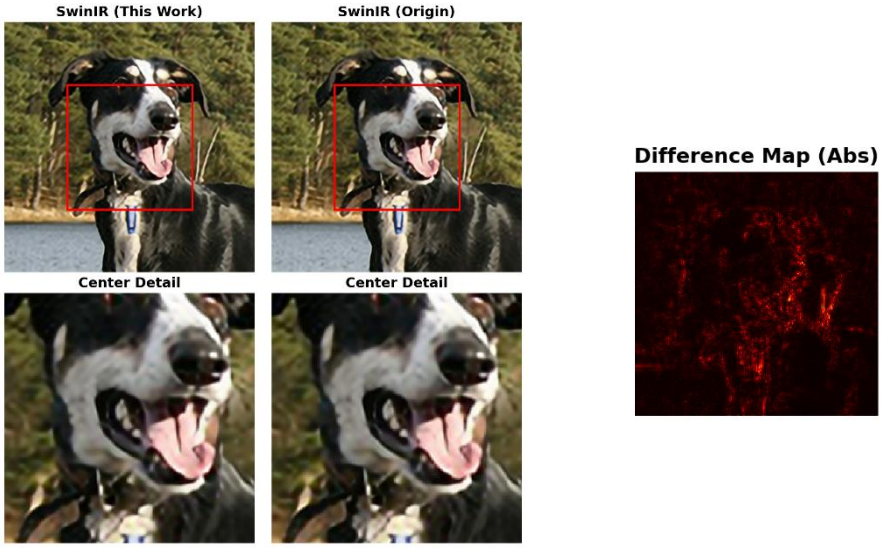
第四章 專題結果展示

4.1 本專題模型訓練結果

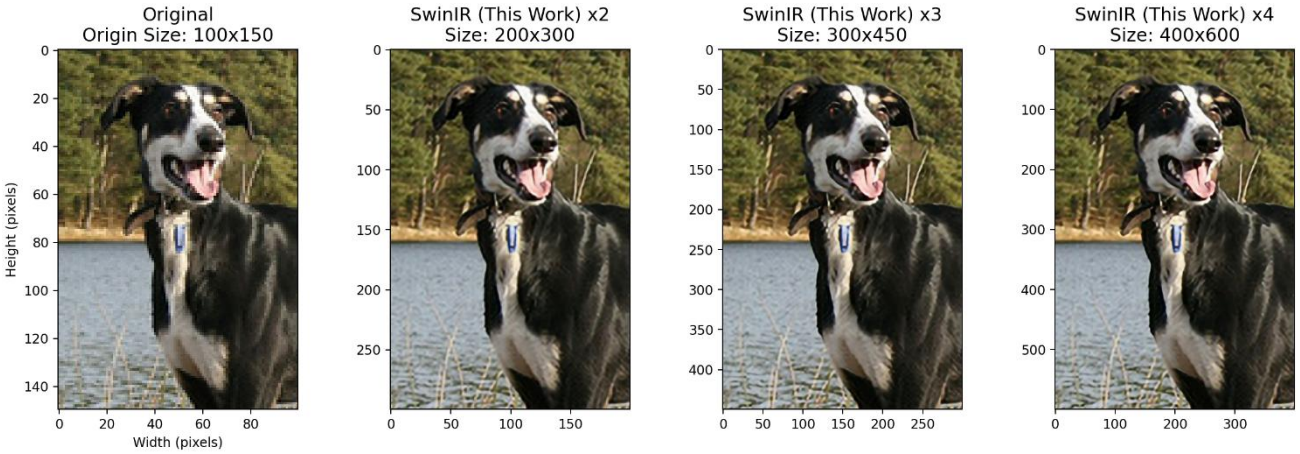
本次專題的 SwinIR 將 STL 的 LayerNorm 改為 BatchNorm，並在 MLP 增加 BatchNorm 進行訓練，與原 SwinIR 進行比較，如表三所示，也可以從 PSNR、SSIM、Differnce Map 上的表現看到本次專題的 SwinIR 能夠維持接近與原有 SwinIR 架構接近的影像品質。

Training Dataset	Model	Scale	# of Parameter	Validation Dataset									
				Set5		Set14		BSD100		Urban100		Manga109	
				PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
DIV2K	SwinIR (Origin)	X2	11.8 M	38.46	0.9624	34.61	0.9260	32.55	0.9043	33.95	0.9433	40.02	0.9800
	SwinIR (This Work)		2450291	37.495	0.9585	33.529	0.9175	31.895	0.8931	32.84	0.9218	38.54	0.9688
	SwinIR (Origin)	X3	11.8 M	35.04	0.9322	31.00	0.8542	29.49	0.8150	29.90	0.8841	35.28	0.9543
	SwinIR (This Work)		2644147	33.508	0.9185	30.044	0.8377	28.909	0.8015	28.94	0.8633	33.89	0.9459
	SwinIR (Origin)	X4	11.8 M	32.72	0.9021	28.94	0.7914	27.83	0.7459	27.07	0.8164	31.67	0.9226
	SwinIR (This Work)		2598003	31.844	0.8878	28.408	0.7775	27.25	0.7125	27.00	0.8132	30.50	0.9148

表三、專題模型訓練結果與 Swin IR 模型訓練結果比較



圖十五、專題模型推論結果與 SwinIR 模型推論結果比較



圖十六、專題模型推論結果

Classification Model	Evaluation Metrics	Origin(X1)	Bicubic(X2)	SwinIR (Origin X2)	SwinIR (This Work X2)
Yolo v5	Top 1	0.7151	0.7148	0.6985	0.6980
	Top 5	0.9022	0.9015	0.8936	0.8933
	Precision	0.7200	0.7203	0.7065	0.7057
	Recall	0.7151	0.7148	0.6985	0.6980
	F1	0.7117	0.7114	0.9550	0.6944

表四、YOLO v5 在不同放大方法下的推論結果比較(X2 放大倍率)

4.2 硬體推論時間

Inference Dataset	Evaluation Metrics	Scale		
		X2	X3	X4
Imagenet (Random 1000)	GPU Inference Time	197.98 s	460.84 s	840.64 s
	CPU Inference Time	1249.94 s	2791.67 s	5011.53 s
	GPU FPS	2.04 s	5.43 s	8.69 s
	CPU FPS	12.5 s	40.9 s	55.87 s

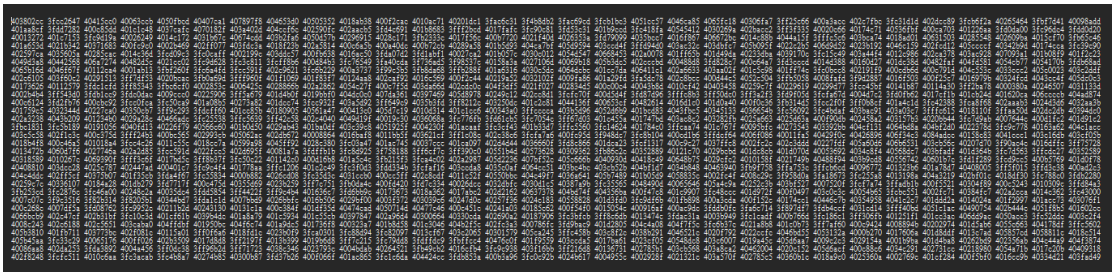
表五、GPU 與 CPU 在 SwinIR 模型的推論速度

4.3 RTL 電路模擬結果

以下結果為使用 RTL 進行模組設計並利用 Testbench 進行模擬得出的結果，由於使用 RTL 所以會出現速度較慢的情況，但透過浮點數設計可以保證 RTL 輸出的精準度較高。

I. Conv Unit

Conv Unit 模擬結果如表三所示，由於使用浮點數進行運算以及只有乘法與加法的原因，精準度可以到相當高的標準，另外，因為使用 Stream-Based Line buffer 的緣故可以在輸入進行的期間同時進行計算，所以 Latency 的時間也相當低。由於模組皆是使用浮點數運算，因此需要較多的運算元才可以完成輸出，在加法器上需要九個運算元，乘法器需要六個運算元，因此在表六中看到 Throughput 會較高。



圖十七、Conv Unit 模擬結果(W=32, H=32, K=3, S=1)

Latency 計算方式如下:

$$Latency = 1 + K \times K + H_{conv} \times W_{conv} + 1(next\ clk\ output), set_lu = 0$$
$$= 1 + K \times K + H_{conv} \times W_{conv} + 1 + 1(LeakyReLU), set_lu = 1$$

W, H, K, S	Accurancy	Latency
32, 32, 1, 1	99.9999%	23.341 us
32, 32, 3, 1	99.9999%	23.523 us
32, 32, 3, 2	99.9999%	23.523 us
32, 32, 3, 3	99.9999%	23.523 us
32, 32, 3, 4	99.9999%	23.523 us
32, 32, 4, 1	99.9999%	23.682 us
32, 32, 5, 1	99.9999%	23.886 us
32, 32, 5, 2	99.9999%	23.886 us
32, 32, 5, 5	99.9999%	23.886 us
32, 32, 5, 6	99.9999%	23.886 us
32, 32, 31, 1	99.9999%	45.159 us
32, 32, 32, 1	99.9999%	46.591 us
32, 16, 3, 1	99.9999%	11.886 us
16, 32, 3, 1	99.9999%	11.886 us
16, 32, 4, 1	99.9999%	11.682 us
16, 31, 3, 1	99.9999%	11.523 us

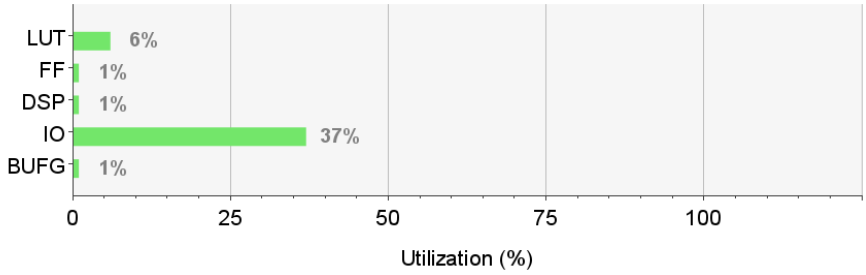
表六 Conv Unit 模擬結果(1 個C_{conv})

Throughput	Clock Speed
24.6 GOPS	44 MHz

表七、Conv Unit 量測結果(1 個C_{conv})

W, H, K, S	Accurancy	Latency
32, 32, 3, 1	99.9999%	23.354 us

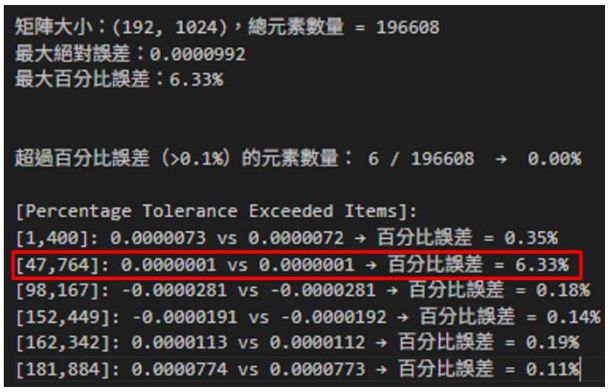
表八、Conv Unit + LeakyReLU Fused Layer 模擬結果



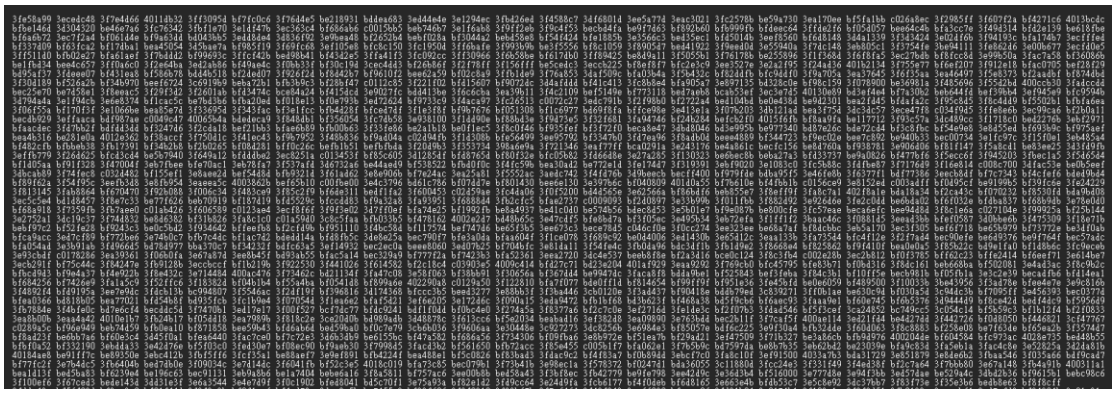
圖十八 Conv Unit Utilization

II. BatchNorm

BatchNorm 在計算上使用了浮點數的除法與根號運算，在速度上相對其他的元件較慢，且在數據量大的情況下會出現資料的偏差，但誤差的數值實際上是計算太過接近而導致的問題。



圖十九、BatchNorm 與 Golden Pattern 比較結果



圖二十、BatchNorm 模擬結果(B=2, H×W=1024, C=96)

Latency 的計算如下:

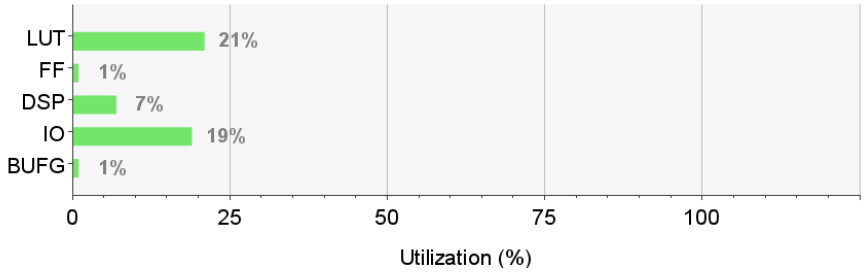
$$Latency = (3 \times H_{Batch} \times W_{Batch} \times B_{Batch} + 1) \times C$$

B, H×W, C	Accurancy	Latency
2,1024,96	93.57%	65.540 ms
2,16, 4	99.9999%	14.370 us
2,16, 3	99.9999%	10.778 us
2,3, 3	99.9999%	2.111 us
1, 3, 3	99.9999%	1.111 us

表九、BatchNorm 模擬結果

Clock Speed
27 MHz

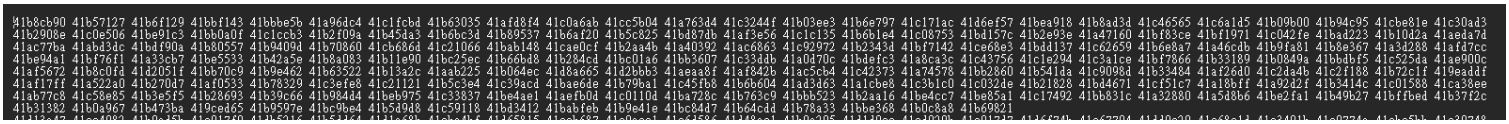
表十、BatchNorm 量測結果



圖二十一、BatchNorm Utilization

III. FC Unit

FC Unit 結果如表三所示，因為只有加法、乘法的原因，精準度可以到相當高的標準，在本次專題的電路中是最快的元件。



圖二十二、FC Unit 模擬結果(W=1024, H=96, F=1024, E=192)

Latency 的計算如下:

$$Latency = 1 + (C_{in} + C_{in} \times C_{out}) \times H_{fc} \times W_{fc} + 1$$

WxH, C_{in}, WxH, C_{out}	Accurancy	Latency
1024, 96, 1024, 192	99.9999%	18.972 ms
1024, 192, 1024, 96	99.9999%	19.701 ms
64, 96, 64, 192	99.9999%	1.186 ms
32, 32, 32, 64	99.9999%	66.562 us
32, 64, 32, 32	99.9999%	67.586 us
16, 32, 16, 16	99.9999%	8.709 us
16, 16, 16, 32	99.9999%	8.450 us
15, 30, 15, 15	99.9999%	7.202 us
15, 15, 15, 30	99.9999%	6.977 us
1, 1, 1, 1	99.9999%	0.004 ns
1024, 192, 1024, 288	99.9999%	56.819 ms

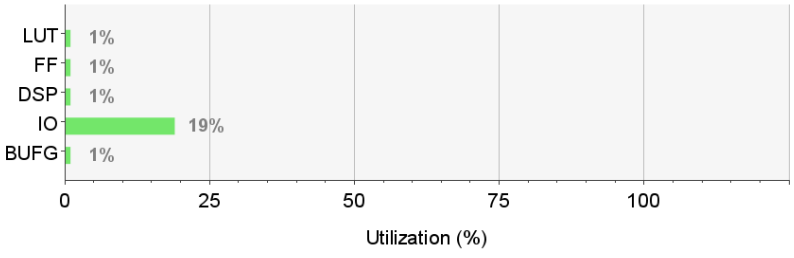
表十一、FC Unit 模擬結果

Clock Speed
88.7 MHz

表十二、FC Unit 量測結果

W, H, F, E	Accurancy	Latency
1024, 96, 1024, 192	99.9999%	18.972 ms

表十三、FC Unit +GeLU Fused Layer 模擬結果



圖二十三 FC Unit Utilization

IV. Residual

Residual 結果如表三所示，因為只有加法的原因，精準度也可以到相當高的標準，進行 pipeline 時也可以更加有效。



圖二十四、Residual 模擬結果(W=32, H=32)

Latency 的計算如下:

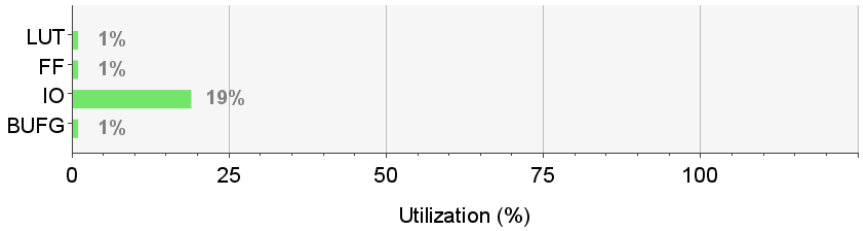
$$Latency = H_{Res} \times W_{Res} + 1$$

W, H	Accurancy	Latency
32, 32	99.9999%	0.512 us
64, 64	99.9999%	4.096 us
63, 63	99.9999%	3.969 us
63, 33	99.9999%	2.079 us
1, 1	99.9999%	0.001 us

表十四、Residual 模擬結果

Clock Speed
58.5 MHz

表十五、Residual 量測結果



圖二十五、Residual Utilization

V. SA Unit

SA Unit 的表現如表十二所示，只有在輸入大量的情況會遇到精準度下降的問題，可以看到即便有 Self-Attention 這樣的大量計算，SA Unit 也可以達到不錯的速度。

```
3eb6828a 3f1b01bc 3f06ba9e 3f2150a3 3f346105 3f1c3d62 3eed8533 3f11a293
3ef6bcee 3f1435c9 3f1d9bea 3f14e8cd 3f24b33a 3f2c7e91 3eb9ab10 3f0254fd
3eb2489e 3f1c2039 3f000d57 3f26302c 3f2db149 3f1b899b 3ed8cd82 3f16a7ec
3ec58f1c 3f184fe4 3f050dcf 3f215c01 3f245be5 3f1dc58d 3ec39e84 3f10241b
3eb955ff 3f19f7f7 3efade51 3f1c5bb4 3f2983e4 3f155358 3f02f990 3f168ff1
3ed8027e 3f096238 3f154248 3f049187 3f265be0 3f19d536 3ebac648 3edbd943
3ebd1b9b 3f1156f3 3f08feaa 3f1310b3 3f315d83 3f0ee269 3ed992a7 3f01ee90
3ed3a6c4 3f134a9a 3f09f25b 3f13307a 3f204a53 3f1b8612 3ed758fe 3f070e78
```

圖二十六、SA Unit 模擬結果(W=8, H=8, R=2)

Latency 的計算如下:

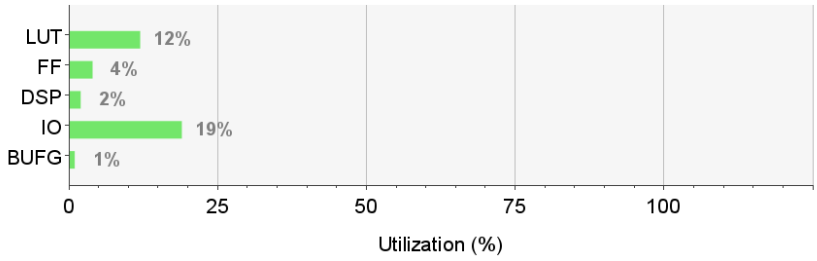
$$Latency = ((1 + (W_{SA} + (W_{SA} + 1) \times H_{SA}) \times H_{SA} + 1) + (H_{SA} \times H_{SA} + T + 1 + H_{SA})) + ((W_{SA} \times H_{SA}) \times H_{SA} + 1)$$

W, H, R	Accurancy	Latency
64, 16, 2	99.85%	33.362 us
64, 16, 3	99.99%	33.362 us
64, 16, 4	99.9999%	33.362 us
8, 8, 2	99.9999%	1.226 us
8, 8, 3	99.9999%	1.226 us
8, 8,	99.9999%	1.226 us
3, 4, 2	99.9999%	0.137 us

表十六、SA Unit 模擬結果

Clock Speed
61.3 MHz

表十七、SA Unit 量測結果



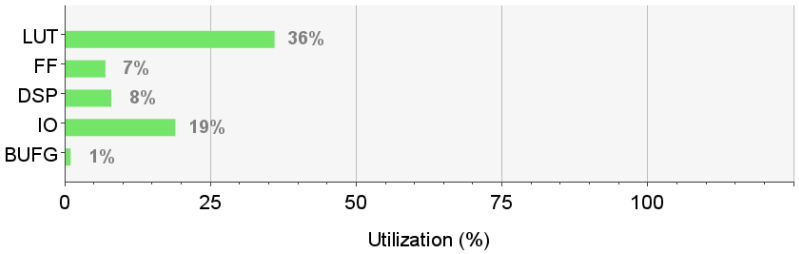
圖二十七、SA Unit Utilization

VI. Control Unit

本次專題中進行全部子電路控制的重要元件，這邊測量精準度的部份因為 Testbench 在輸入上會超過限制所以並未進行測試，目前只有 Latency 的測量，以及 Clock Speed。

Clock Speed
27.5 MHz

表十八、Control Unit 量測結果



圖二十八、Control Unit Utilization

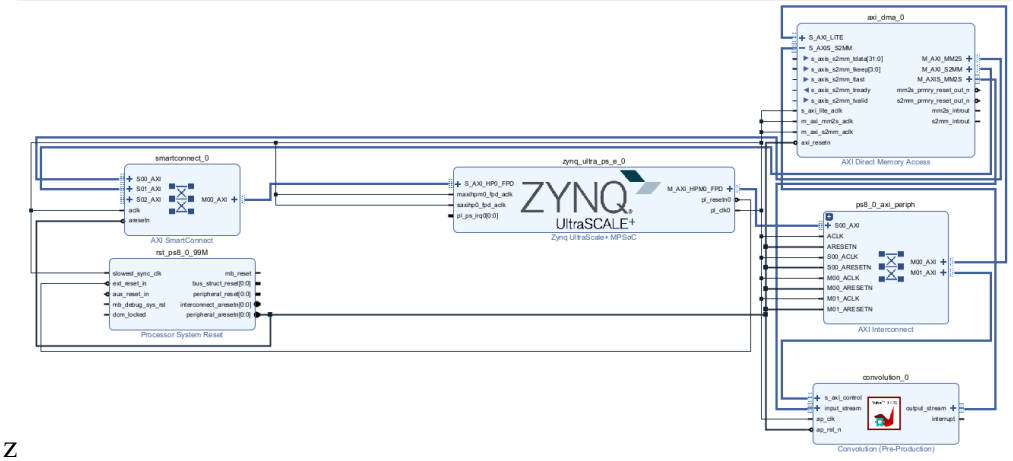
4.4 HLS 電路模擬結果

I. Conv Unit (Row Stationary)

本次的專題也進行了 Row Stationary 的 HLS 模擬，將在後文與 Stream-Based Linebuffer 的效能進行比較。

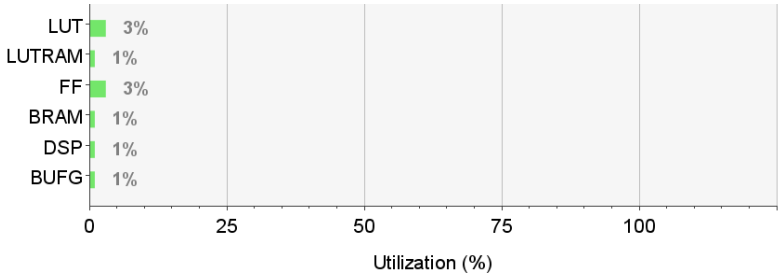
Clock Speed
162.52MHz

表十九、Conv Unit (Row Stationary) HLS 量測結果

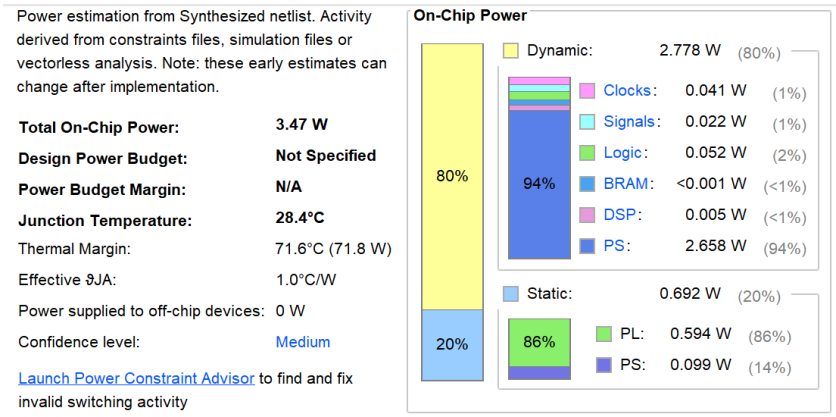


z

圖二十九、Conv Unit (Row Stationary) Block Design



圖三十、CONV Unit (Row Stationary) Utilization



圖三十一、CONV Unit (Row Stationary) Power Analysis

II. Conv Unit(Stream-Based Linebuffer)

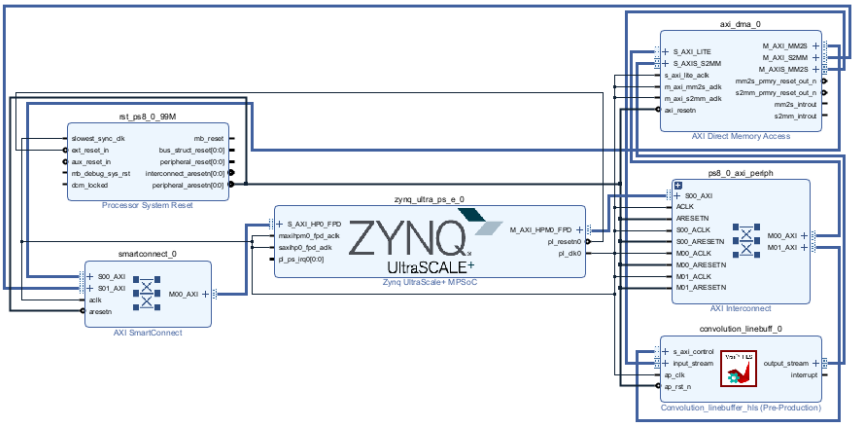
可以從這裡的量測看到，雖然 Latency 與 Utilization 低於 Row Stationary，但是 Stream-Based Linebuffer 在功耗上較低且時脈速率較高可以達成較好的應用。

Convolution	Clock Speed	DSP	LUT	FF	BRAM	Power
Row Stationary	162.52 MHz	1%	4 %	3%	1%	3.47 W
Stream-Based Linebuffer	181.1 MHz	1%	3 %	3%	1%	3.444 W

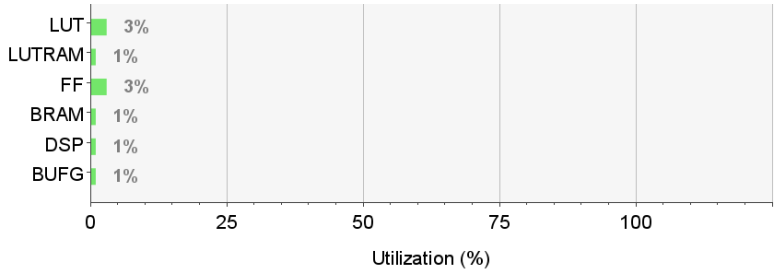
表二十、Convolution 比較結果

Clock Speed
181.1 MHz

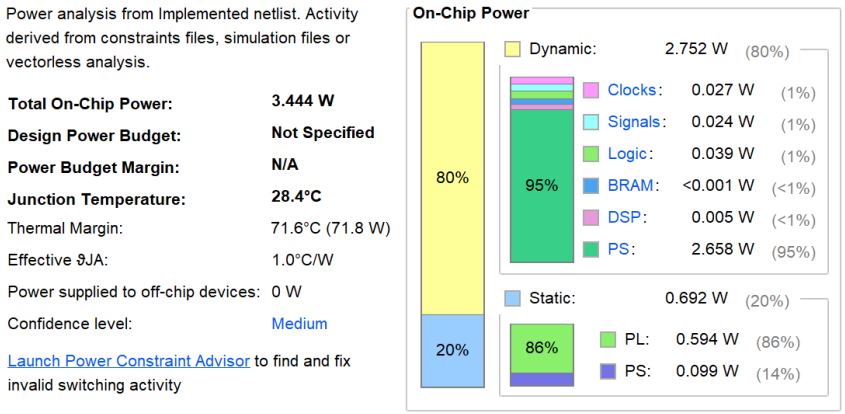
表二十一、Conv Unit Stream-Based Linebuffer)量測結果



圖三十二、Conv Unit (Stream-Based Linebuffer) Block Design



圖三十三、CONV Unit (Stream-Based Linebuffer) Utilization

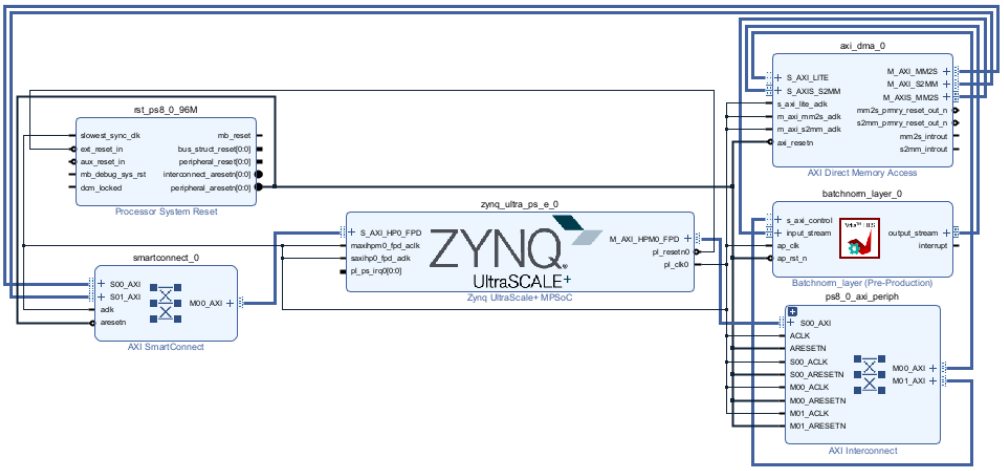


圖三十四、CONV Unit (Stream-Based Linebuffer) Power Analysis

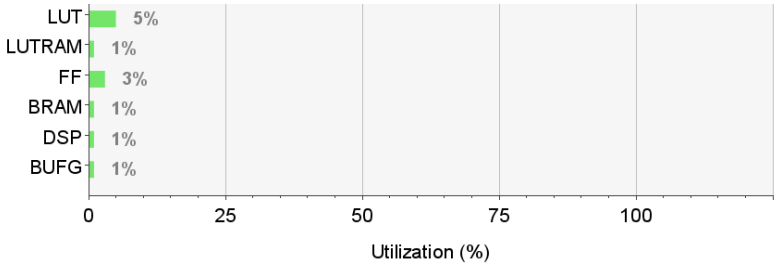
III. BatchNorm

Clock Speed
146.8MHz

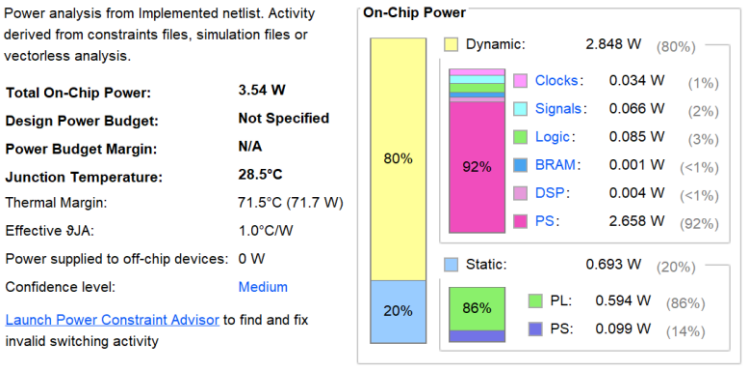
表二十二、BatchNorm HLS 量測結果



圖三十五、BatchNorm Block Design



圖三十六、BatchNorm Utilization

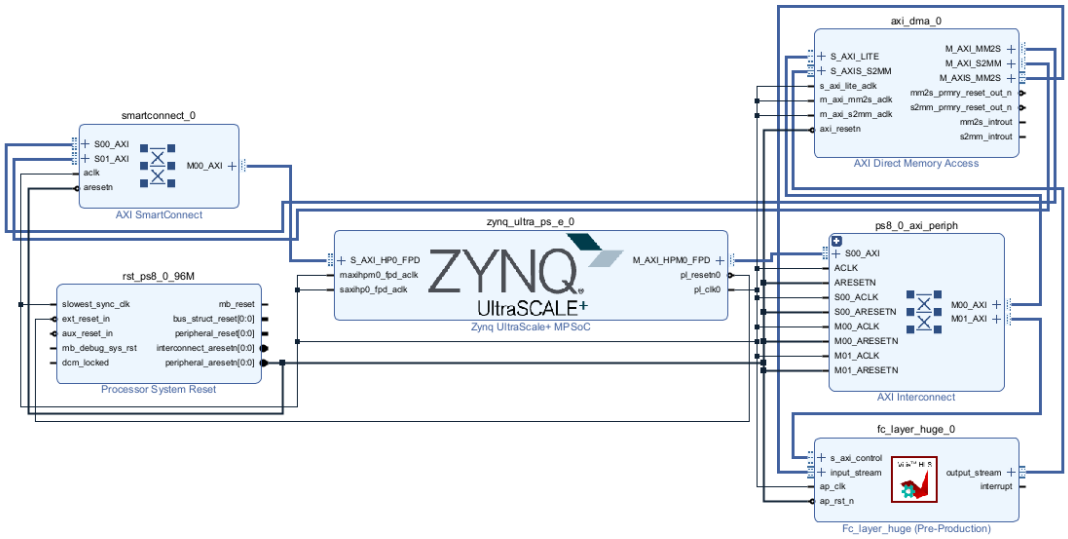


圖三十七、BatchNorm Power Analysis

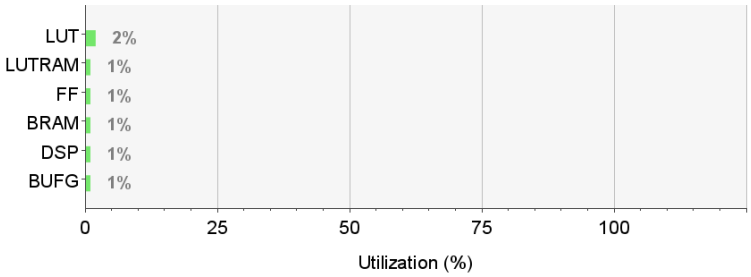
IV. FC Unit

Clock Speed
188.6 MHz

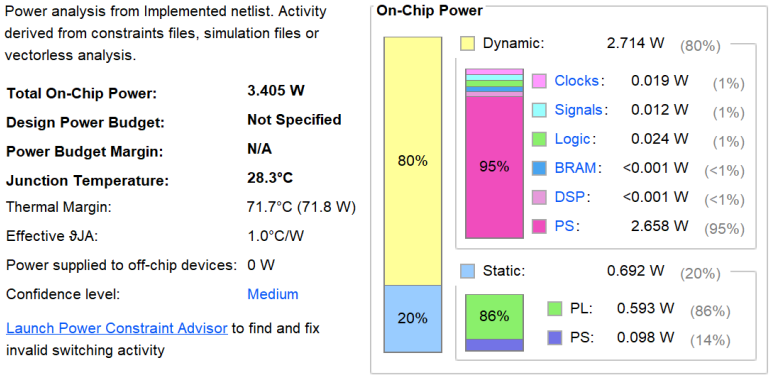
表二十三、FC Unit HLS 量測結果



圖三十八、FC Unit Block Design



圖三十九、FC Unit Utilization

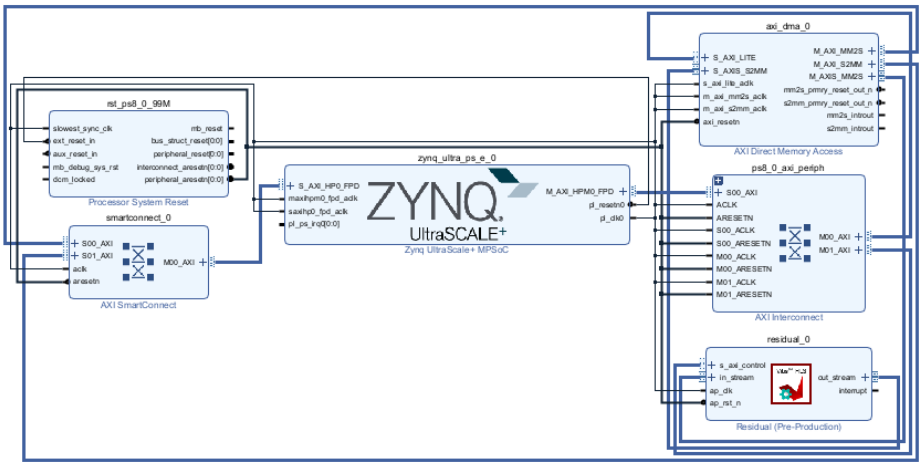


圖四十、FC Unit Power Analysis

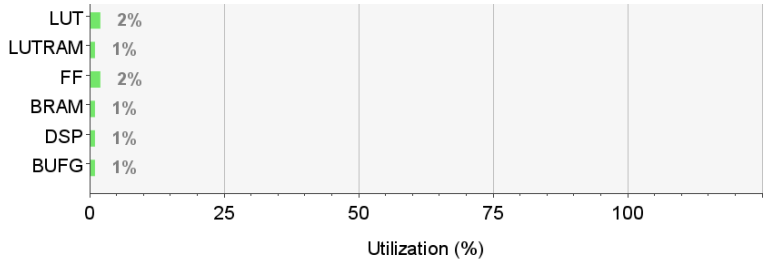
V. Residual

Clock Speed
178.3 MHz

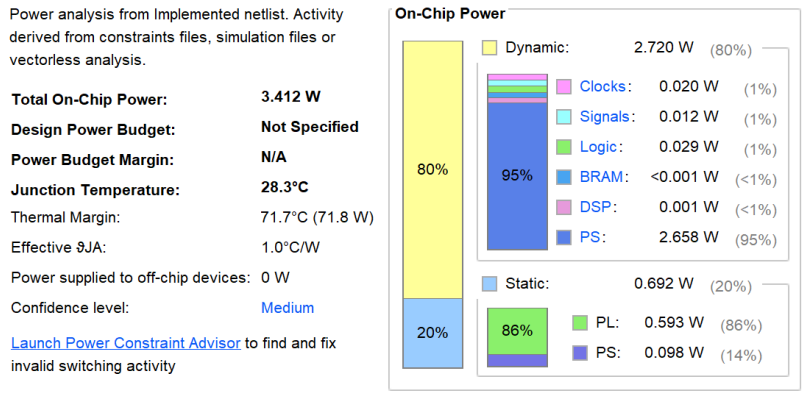
表二十四、Residual HLS 量測結果



圖四十一、Residual Block Design



圖四十二、Residual Utilization

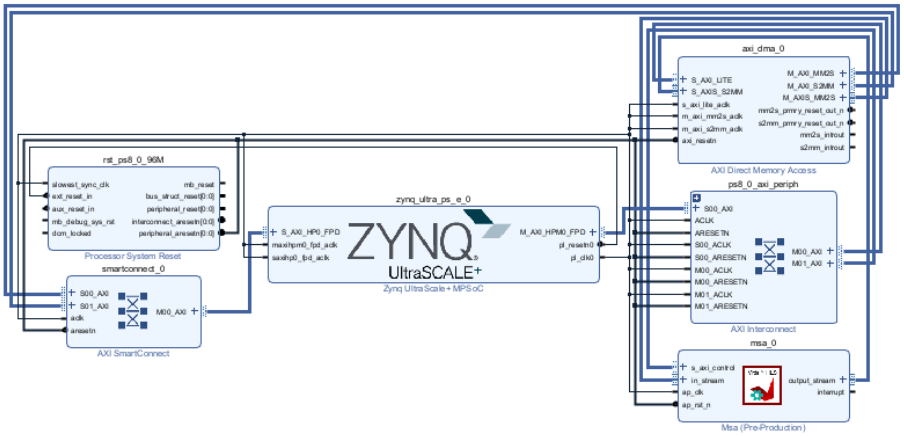


圖四十三、Residual Power Analysis

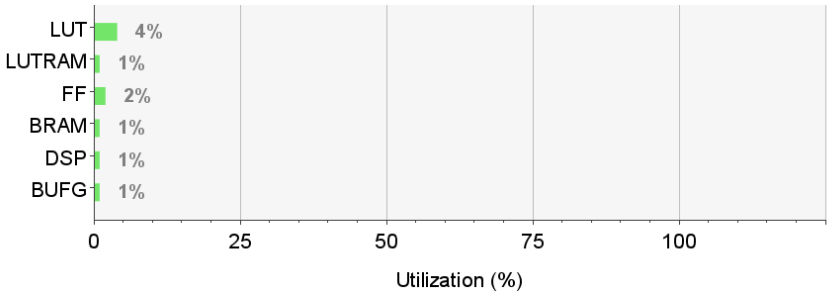
VI. SA Unit

Clock Speed
200.9 MHz

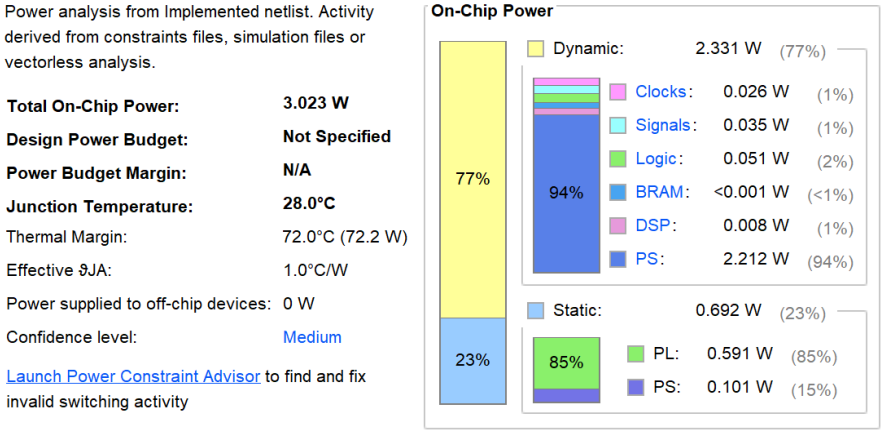
表二十五、SA Unit HLS 量測結果



圖四十四、SA Unit Block Design

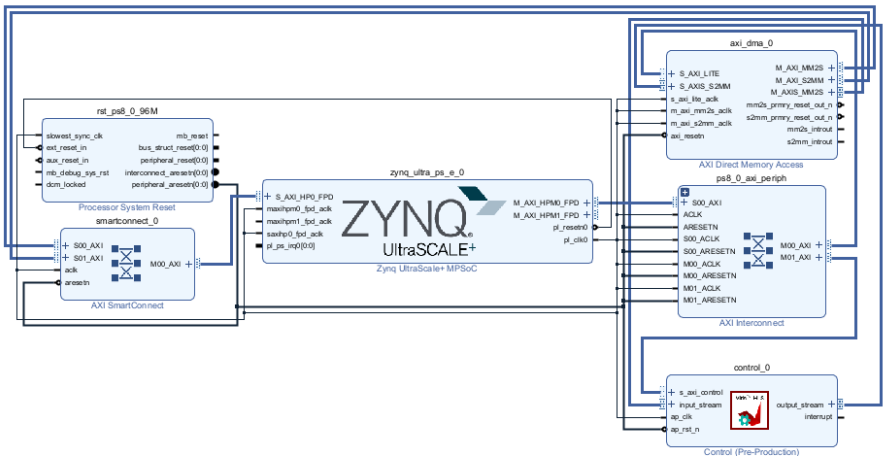


圖四十五、SA Unit Utilization



圖四十六、SA Unit Power Analysis

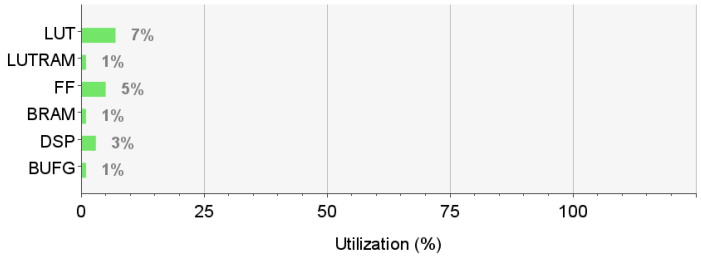
VII. Control Unit



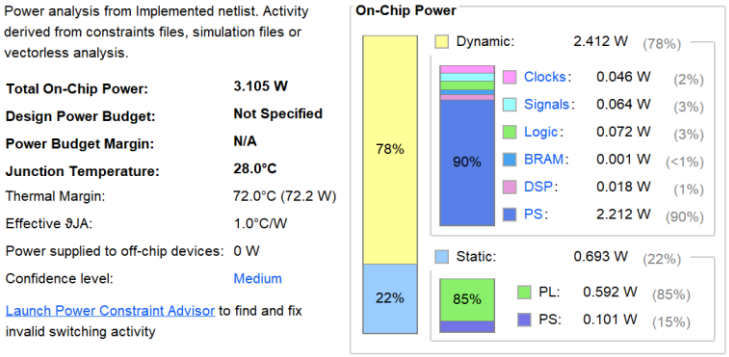
圖四十七、Control Unit Block Design

Clock Speed
154.8 MHz

表二十六、Control Unit HLS 量測結果



圖四十八、Control Unit Utilization



圖四十九、Control Unit Power Analysis

Device	Light QFSRCNN[5]	定點數最佳化 SwinIR 超解析硬體加速器
FPGA	Kintex-7 XC7K410T	ZCU106-Zynq UltraScale+ MPSoC
Precision	FP 13	INT32
DSP	1512 (98 %)	47 (2.72 %)
LUT	167 K (41 %)	15K (6.55 %)
FF	158 K (31 %)	23K (5.07 %)
BRAM	188 (24 %)	3 (0.96 %)
Troughtput	780 GOPS	30.2 GOPS
Clock Speed	130 MHz	154.8 MHz

表二十七、定點數最佳化 SwinIR 超解析硬體加速器與 QFSRCNN 結果比較表

第五章 結論與未來展望

本次的設計不只挑戰了高 PSNR 的 SR 硬體加速器，也利用 Stream-based Line buffer 達成高效輸入的策略，與其他 SR 設計不同本次設計考量了高擴展性的硬體設計，使用上可以根據需求自由調整運算速度、精準度、功耗需求，又或者是將效率最大化挑戰更高的 PSNR。本次的設計讓 SwinIR 模型能在嵌入式平台上即時運作，並超越現有的 SR 硬體加速器，也能突破 CPU、GPU 的硬體的 SwinIR 推論表現。預計在未來挑戰進行定點數的量化，進一步減少硬體資源消耗，並參考[9]在包含非線性單元計算的架構上，例如:Softmax、GELU，進行計算式的近似，將算式簡化為可以使用，並透過定點數與高精準度的量化策略進行更高 PSNR 的挑戰，預期減少浮點數帶來的資源消耗、計算時間、功耗的同時加快計算速度，帶來更好的效益。

參考資料

[1] Jingyun Liang, Jiezhang Cao, Guolei Sun, Kai Zhang, Luc Van Gool, Radu Timofte. SwinIR: Image Restoration Using Swin Transformer. arXiv:2108.10257. 2021.

[2] Jacob Shermeyer, Adam Van Etten. The Effects of Super-Resolution on Object Detection Performance in Satellite Imagery. arXiv:1812.04098. 2018

[3] MICHAŁ KOZIARSKI, BOGUSŁAW CYGANEK. Impact of Low Resolution on Image Recognition with Deep Neural Networks: An Experimental Study. amcs-2018-0056. 2018

[4] Wenqin Luo, Patrick Hung, Shiqi Wang, Ray C.C. Cheung. Image Super-Resolution and FPGA Hardware Design. ICSPCC59353.2023.10400225. 2023.

[5] J. -W. Chang, K. -W. Kang and S. -J. Kang. An Energy-Efficient FPGA-Based Deconvolutional Neural Networks Accelerator for Single Image Super-Resolution. in *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 1, pp. 281-295, Jan. 2020, doi: 10.1109/TCSVT.2018.2888898. 2018

[6] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. arXiv preprint arXiv:2103.14030. 2021.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin. Attention Is All You Need. arXiv:1706.03762. 2017

[8] Hyeong-Ju Kang. AoCStream: All-on-Chip CNN Accelerator With Stream-Based Line-Buffer Architecture. arXiv:2212.11438. 2022

[9] Zhiyang Liu, Pengyu Yin, Zhenhua Ren. An Efficient FPGA-Based Accelerator for

Swin Transformer. arXiv:2308.13922. 2023
[10] Chao Dong, Chen Change Loy, Kaiming He, Xiaoou Tang. Image Super-Resolution Using Deep Convolutional Networks. *IEEE transactions on pattern analysis and machine intelligence* 38.2 (2015): 295-307.
[11] Christian Ledig, Lucas Theis, Ferenc Huszar, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, Wenzhe Shi. Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network. Proceedings of the IEEE conference on computer vision and pattern recognition. 2017.
[12] Y. -H. Chen, T. Krishna, J. S. Emer and V. Sze. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks. in IEEE Journal of Solid-State Circuits, vol. 52, no. 1, pp. 127-138. 2017.